NOVA IMS INFORMATION MANAGEMENT SCHOOL

When there is no right answer

Evaluating GenAI applications — part two of two

Session 6 · Evaluation II

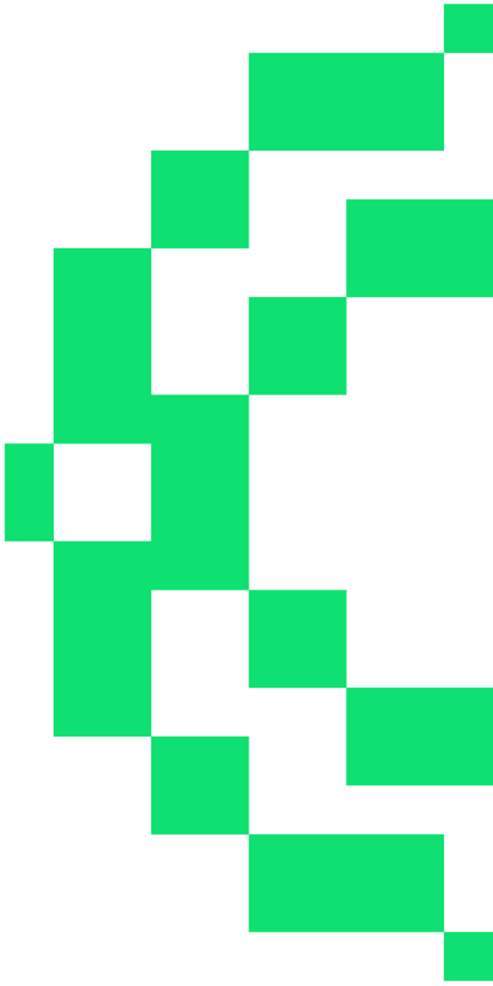
Hugo Silva, September 2026

Tonight

An hour on agents judging agents, then an hour on your build.

1 · Using an LLM as a judge, and checking that it works.
2 · Measuring the retrieval and the steps, not just the answer.
3 · What public benchmarks can and cannot tell you.

Then: one hour on your project, with me in the room.



What code caught, and what it could not

Twenty failures. Thirteen of them were an equals sign.

THE 13 THAT CODE CAUGHT

THE CHECK	WHAT IT MEANS	N
`quote in ticket`	the evidence quote is a substring of the ticket	4
`len(labels) == 1`	exactly one label, and nothing else in the field	2
`label in LABELS`	the category is one you defined	2
`set(pred) == set(gold)`	the right category when two could apply	5
four one-liners	no model, no cost, no disagreement	13

AND IN YOUR PROJECT

output length · a keyword that must appear · a forbidden word · response time · does it parse · is every figure quoted also in the table

THE 7 IT COULD NOT — ALL ONE FAMILY

`urgency inflated by emotional wording  7`

The company's rule: **high only if the customer is blocked from working now.** Tone, capitals and politeness are not evidence.

TICKET 20

"Honestly terrible service. All I want is the VAT number changed on our receipts."

the classifier said **high**

Whether somebody is blocked is a reading of the ticket, not a property of its characters. No regular expression separates this from a real outage. Rewriting the prompt moved the number; it never made the check writable.

read → cluster → name → fix the prompt → re-score

That loop is last week. Tonight it runs a second time — on the grader itself.

Session 5, slides 8–21: read twenty outputs, cluster them, name the families, fix the prompt, then score against a frozen golden set.

Last week, and where it stops

A category has one right answer. A reply has a hundred good ones.

LAST WEEK, IN FOUR STEPS

look › name › freeze › score

CLASSIFY A TICKET

done — last week

One right category. Grader: `==`. Correct is a **fact**, and the fact is in your golden set.

DRAFT THE FIRST REPLY

tonight

A hundred acceptable replies, and no list of them anywhere. Grader: a **rubric**, applied by something that can read. Correct is a **judgement**.

AN AGENT THAT LOOKS THINGS UP AND ACTS

tonight

The answer can be right and the path wrong — it read the wrong document, or called the refund tool twice. Correct is a **process**.

Three shapes, three graders. Decide which one your project is before you write a line of evaluation code — the answer changes everything downstream.

01.
The judge

Scoring what has no answer key

A rubric is a specification

Criteria become dimensions. Each one is a separate question.

LAST WEEK'S PAIR, CARRIED FORWARD

failure mode — what you saw go wrong, repeatably.
evaluation criterion — the dimension you check every time.

A rubric is your criteria, written so somebody else could apply them.

TWO WAYS TO ASK FOR THE SAME JUDGEMENT

WHAT PEOPLE WRITE

"Rate the helpfulness of this reply from 1 to 5."

- helpfulness is undefined
- the scale hides *which* dimension failed
- nothing a second person could apply the same way

WHAT A RUBRIC IS

1 · answers the question asked — pass if the customer's actual question is addressed. A correct answer to a different question is a fail.

2 · grounded in the policy — pass if every factual claim can be pointed at in SP-04. "I checked and you are eligible", with no clause behind it, is a fail.

↑ each one traces back to a failure family from last week

RULE 1 — ONE JUDGE CALL PER DIMENSION

"grade each dimension with an isolated LLM-as-judge rather than using one to grade all dimensions"

RULE 2 — GIVE IT A WAY OUT

"give the LLM a way out, like providing an instruction to return 'Unknown' when it doesn't have enough information"

Anthropic Engineering, "Demystifying Evals for AI Agents" (9 Jan 2026).

The judge prompt, in full

A lyric assistant, judged on one dimension at a time

```
You are grading ONE quality of a song lyric. Nothing else.          ①
Do not comment on style, length, or how much you like it.

DIMENSION — Brief adherence:                                        ②
  PASS if the lyric uses the brief's setting, point of view
       and emotional turn.
  FAIL if any of the three is absent or contradicted.
  UNKNOWN if the brief does not state one of them.                  ③

BRIEF:      {brief}                                                 ④
LYRIC:      {lyric}

First quote the line that decides it. Then give the verdict.       ⑤
Quote exactly; if no line decides it, return UNKNOWN.

Return JSON only:                                                   ⑥
{"evidence": "...", "verdict": "PASS|FAIL|UNKNOWN", "why": "..."}
```

DELIBERATELY ABSENT

~~rate the quality 1–10 · also check the rhyme and the tone · use your judgement~~

EVERY BLOCK IS A DECISION

① **one dimension per call**
Two dimensions in one prompt gives you one number and no idea which half failed.

② **the criterion, defined**
Same four parts as a label definition: what passes, what fails, and the boundary.

③ **the way out**
Without UNKNOWN a judge invents a verdict rather than admitting the brief was silent.

④ **inputs, labelled and separated**
Session 3. The judge must not confuse the brief with the thing being judged.

⑤ **critique before label**
The anti-hallucination move: quote first, decide second, so the reason cannot be written backwards.

⑥ **structured output**
Verdicts you can count. Agreement with a human is arithmetic, not reading.

The evidence field is the whole trick. A judge that must quote a line before it decides cannot invent the reason afterwards.

Same four parts as last week's classifier prompt: a role, a defined criterion, an abstain option, and a structured output. The judge is an application; it is engineered like one.

Align it to a human, or skip it

A judge you have not measured is a number, not evidence

1. **one domain expert**
   "one (maybe two) key individuals whose judgment is crucial". Not a committee.
2. **they label pass/fail, with a written critique**
   The same shape as last week's error analysis.
3. **their critiques become the judge prompt**
   You are not writing a judge. You are transcribing one.
4. **measure agreement on held-out cases**
   As **TPR and TNR**, both of them — a judge that passes everything scores 100% on one and zero on the other.
5. **disagree → rewrite → re-measure**
   Three rounds is usually enough to agree with the expert more than nine times in ten — Husain reports >90% agreement within three iterations.

THE MEASUREMENT, IN TWO NUMBERS

Take the human's verdict as the truth, and **PASS as the positive class**.

TPR — of the replies the human passed, how many did the judge also pass?
TNR — of the replies the human failed, how many did the judge also fail?

You need both. A judge that passes everything scores TPR 100% and TNR 0.

"Focus on achieving high True Positive Rate and True Negative Rate with your judge on a held out labeled test set."

Skip step 4 and you have replaced an unmeasured application with an unmeasured judge.

Husain — hamel.dev/blog/posts/llm-judge/ — "Use LLMs to scale what you've learned, not to avoid looking at data."

Demo 1 — Build a judge, and check it

Ten minutes. Setup: one chat window, one Word file.

LIVE DEMO

Kestrel judge demo.docx on Moodle — rubric, replies, prompts

1. Ask the chat which of replies 4 and 7 is better. Ask again in a fresh chat with the two swapped. Note whether the winner changed.

2. Now score replies 1, 2 and 3 yourself, against the two criteria, in the table on page 1. One line of critique each, before any model sees them.

3. Paste the judge prompt, then the policy and the replies. Compare its verdicts on 1, 2 and 3 with yours, and count the agreements.

WHAT TO WATCH FOR

- Step 1 has no right answer, and the model supplies one anyway. A verdict that changes with the order was partly about the order.
- Step 2 is the only step that produces something you can measure the judge against. It is also the only step with no model in it.
- Step 3 is the only step that tells you the judge is usable. Everything you do after it assumes it is.
- Scoring each reply against a fixed rubric mostly removes the effect step 1 just showed you.

Zheng et al. (2023) measured step 1 at scale: the judges they tested were consistent on only 65% of swaps, and only GPT-4 cleared 60%.

Three biases, with numbers

Measured on the strongest judge of 2023. Re-measure yours.

POSITION

"a propensity to favor certain positions over others"

65.0%

consistent when the two answers are swapped

30.0% biased to the first · 5.0% to the second. *"Only GPT-4 outputs consistent results in more than 60% of cases."*

VERBOSITY

The repetitive-list attack: the same points restated as a list, no new content.

GPT-4	**8.7%** fooled
GPT-3.5	**91.3%**
Claude-v1	**91.3%**

"all LLMs may be prone to verbosity bias"

SELF-ENHANCEMENT

Judging their own answers, over the human vote:

- GPT-4 **+10** points
- Claude-v1 **+25** points

and the authors' own hedge: *"Due to limited data and small differences, our study cannot determine whether the models exhibit a self-enhancement bias."*

Measured in 2023. Every model in this table is gone. Which is why you measured your own in the demo.

Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena", NeurIPS 2023 Datasets & Benchmarks — arxiv.org/abs/2306.05685

So why does a judge work at all?

Session 4 said models cannot check themselves. Both hold.

SELF-CORRECTION — FAILS

One call. Its own answer. Its own reasoning. **No external reference.**

Huang et al. (ICLR 2024): reflecting with no external signal often makes reasoning **worse** — a model that got an answer wrong usually cannot tell that it did.

vs

LLM-AS-JUDGE — WORKS, CONDITIONALLY

- a separate call
- a different prompt
- an external rubric it did not write
- calibrated against human labels

Four bolts hold it together. Remove any one and you are back on the left.

Huang et al., ICLR 2024 · Kamoi et al., TACL 2024. The four bolts are what separate a judge from self-correction.

What you give the judge decides

Same judge, same questions. Only the prompt changed.

ONE OF THE TEN QUESTIONS

Pens cost €1.20. Marta buys 7 and pays with a €10 note. What is her change?

ANSWER A

"7 × 1.20 = 7.80, so the change is €2.20."

7 × 1.20 = 8.40 — wrong, and confidently shown

ANSWER B

"€1.60."

right, and shows nothing

DEFAULT PROMPT

`"Which answer is better, A or B?"`

The judge never does the arithmetic. It reads two confident answers and picks the one that explains itself better.

wrong 14 of 20

WITH CHAIN-OF-THOUGHT

`"Solve the problem yourself, step by step. **Then** say which answer is better."`

It computes 8.40 before it looks at either answer, and A collapses.

wrong 6 of 20

GIVEN A REFERENCE ANSWER

`"The correct answer is €1.60. Which answer matches it?"`

Now it is a comparison, not a judgement. Almost a string match.

wrong 3 of 20

"although GPT-4 can solve the problem (when asked separately), it was misled by the provided answers."

If a checkable answer exists, give it to the judge.

Better still, do not use a judge at all. That is what last week was for.

Zheng et al. (2023), GPT-4 judging two answers to ten maths questions. Measured in 2023 — the shape of the result is the lesson, not the figures.

Which grader, for which failure?

One row per failure family from your own error analysis

FAILURE FAMILY — THE FIRST FIVE ARE LAST WEEK'S TWENTY	N	GRADER	WHY
the evidence quote is not in the ticket	4	**code**	substring test — exact, free, instant
more than one category, or reasoning mixed in	2	**code**	it parses as one label, or it does not
answered when it should have abstained	2	**code**	membership test against your own list
the wrong category when two could apply	5	**code**	the gold label is known — compare the two sets
urgency inflated by emotional wording	7	**judge**	needs a rubric, and a calibrated one
the reply answers a different question	tonight	**judge**	no key exists; a rubric dimension does
a claim not supported by the policy	tonight	**judge**	and measure the retrieval too — next section

AND THE HUMAN? **Not a row in this table.** A human writes the label definitions, the golden set and the rubric — **the answers the other two are scored against**. It is not a grader you run on every build. If you cannot write the rubric, there is nothing to delegate.

Build a judge only for rows you will re-run. A question you will ask once is cheaper to answer by reading.

The table is the deliverable, not these rows — fill it from your own error analysis.

02.
Machinery

Grade the trace, not the answer

The answer was fine. The search was not.

Session 3's wrong answer, diagnosed and then scored

THE ANSWER

THE CUSTOMER

"We are cancelling our annual plan mid-term. Do we get the unused portion back?"

> *"Yes. Where an agreement is cancelled mid-term we seek a pro-rata credit for the unused portion of the term, so you will be credited for the remaining months."*
>
> cites HB-02 §15.6

Fluent, sourced, and wrong. ***Nothing here looks broken.***

WHAT WAS RETRIEVED

TOP 3, AT k = 3

HB-02 §15.6 | HB-02 §14.2 | HB-02 §19.3

All three share the question's words — *cancelled*, *unused*, *term* — and all three are from the **employee handbook**.

SP-04 §4.2 ranked fourth. Nineteen pages of handbook outvoted a one-page policy.

Here is the bug.

FAITHFULNESS, COMPUTED ON IT

claim 1 — a mid-term cancellation gets a pro-rata credit

✓ **supported by §15.6**

claim 2 — you will be credited for the remaining months

✓ **supported by §15.6**

`2 ÷ 2 = 1.00`

A perfect score on a wrong answer.

§15.6 is about Kestrel's own suppliers, not its customers. The clause that governs this is **SP-04 §4.2** — *the unused portion of the annual term is not refunded* — and it never came back. **Faithfulness measures grounding, never truth.** Session 3's *grounded ≠ true*, with a number attached.

Grade the answer and you will blame the wrong component. The metric that sees this is **context recall**, and it is zero.

Faithfulness = supported claims ÷ total claims, 0–1 — docs.ragas.io. It is computed by a model, so everything on the judge slides applies to it too.

What a trace is

One run, written down. Everything you can grade is in it.

```
{
  "run_id": "t-0417",                                          ①
  "input":  "Our annual plan was closed by mistake on
             3 August and reinstated on 5 August. Can
             we get the unused portion back?",                 ②
  "steps": [
    {"tool": "lookup_account",                                 ③
     "args":   {"account_id": "KS-4466"},
     "result": {"plan": "annual_pro", "paid": 1080.00,
                "closed": "2026-08-03",
                "reinstated": "2026-08-05"}},

    {"tool": "search_policy",
     "args": {"query": "refund unused portion annual plan"},
     "retrieved": [{"id": "SP-04/7.3",  "score": 0.81},      ④
                   {"id": "SP-04/4.2",  "score": 0.78},
                   {"id": "HB-02/15.6", "score": 0.74}]},   ⑤

    {"tool": "issue_refund",
     "args":   {"account_id": "KS-4466", "amount": 900.00},
     "result": {"status": "ok", "ref": "RF-9912"}}
  ],
  "answer": "Because the closure was an error, clause 7.3
             applies and I have refunded the unused
             EUR 900 to your card."                            ⑥
}
```

ONE RUN, WRITTEN DOWN

① **run_id**
One row in your dataset, and the only thing that links a score back to a run.

② **input**
All a Session 5 golden set had. Everything below it is new.

③ **every call, with its arguments**
This is the path — and it is where the side effects live.

④ **chunk *ids*, not chunk text**
Ids are what turn recall into a set comparison. Text does not compare.

⑤ **scores, in rank order**
Context precision needs the ranking, not just the set.

⑥ **the answer**
The part everybody logs — and the only part most teams ever grade.

Log ③ to ⑤, or there is nothing to grade but ⑥. Instrument on day one. You cannot recover a trace you never wrote.

Twenty of these, labelled and scored, are on Moodle as kestrel-traces.jsonl — copy the shape for your own agent.

The golden set grows a shape

Last week a label. Tonight, the trace you expected.

LAST WEEK — ONE INPUT, ONE ANSWER

TICKET	GOLD LABELS
“I was charged twice and now I cannot log in.”	billing, account_access
“Any chance of a dark theme?”	feature_request

Two columns. A label is a word, and comparing two words is an equals sign.

TONIGHT — ONE INPUT, ONE EXPECTED *TRACE*

```
{
  "run_id": "t-0417",
  "must_retrieve": ["SP-04/7.3", "SP-04/4.2", "SP-04/6.2"],
  "must_call":     ["escalate", "issue_refund"],
  "must_not_call_before_escalate": ["issue_refund"],
  "answer_rubric": [
     "states the entitlement and names its clause",
     "states any approval or waiting period that applies"]
}
```

THREE OF THE FOUR ARE LISTS

Comparing lists is **code**. Only `answer_rubric` needs a judge — and that is the one you already know how to calibrate.

WHERE THE WORK ACTUALLY IS

Not the format. **Deciding what should have come back** — which means reading the policy yourself, once, for twenty cases. Still one evening.

Three of the four fields are lists, and comparing lists is code. Only the rubric needs a judge.

Grade each step, not just the end

Seven numbers. Four of them are set comparisons.

ONE RUN, THREE PLACES TO GRADE IT

retrieve › act › answer

ON THE SEARCH

context recall code
did every clause you need come back?

context precision code
of what came back, how much was useful, and how highly ranked?

ON THE PATH

tool call accuracy code
right tool, right arguments?

order and forbidden calls code
did it escalate before refunding? did it call anything twice?

agent goal accuracy judge
did the user get what they came for?

ON THE ANSWER

faithfulness judge
is every claim supported by what came back?

response relevancy judge
does it address the question that was asked?

Four of these seven are set comparisons.

Label the ids and the tool calls, and most of a trace grades itself. The judge is only needed at the end — which is the cheapest place to need one.

Metric names from RAGAS — Es et al. (2024), docs.ragas.io. Names vary between tools; the questions do not.

One trace, scored

The customer is happy. The run failed.

STEP	EXPECTED, AND WHAT HAPPENED	THE NUMBER	VERDICT
search	required `7.3 · 4.2 · 6.2` retrieved `7.3 · 4.2 · 15.6`	recall 2/3 = **0.67**	FAIL
path	expected `escalate → issue_refund` actual `issue_refund`	EUR 900 issued, no escalation	FAIL
answer	three claims, all supported by what was retrieved; it answers the question asked	faithfulness **1.00** · relevancy pass	PASS

§6.2 never came back → the agent had no reason to escalate → it refunded, and said so fluently and correctly

The retrieval failure **caused** the path failure. That is why you fix the earliest failing step, not the loudest one.

`t-0417 | recall 0.67 | path FAIL | faith 1.00 | relevancy PASS | overall FAIL`

An answer-only grader passes this trace. The customer is happy and a control was skipped on a nine-hundred-euro refund.

The €500 threshold is SP-04's own, clause 6.2: any refund above it needs written manager approval before it is issued.

Twenty traces, scored

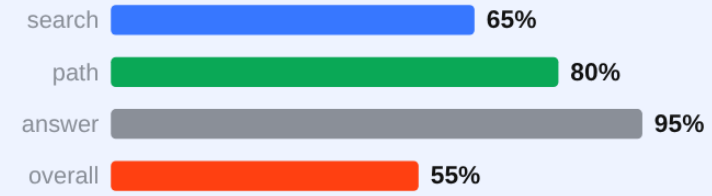
One row per run, and the bottleneck becomes visible.

ONE ROW PER RUN — FIVE PATTERNS, THEN THE WHOLE SET

RUN	SEARCH RECALL	PATH	ANSWER	OVERALL	WHAT WENT WRONG
t-0411	1.00	PASS	PASS	**PASS**	every step clean
t-0416	**0.00**	PASS	PASS	**FAIL**	answered from the wrong document
t-0417	**0.67**	FAIL	PASS	**FAIL**	refund above the 6.2 threshold
t-0419	1.00	PASS	FAIL	**FAIL**	applied 7.3 to a deliberate closure
t-0429	1.00	FAIL	PASS	**FAIL**	refunded a second time
all twenty	**13 / 20 at 1.00**	**16 / 20**	**19 / 20**	**11 / 20**	these are the bars, and the file

*A run passes **search** only at recall 1.00 — a partial retrieval is a miss — and passes **overall** only if all three steps do.*

PASS RATE, ALL TWENTY



The five rows on the left are one example of each pattern, not a sample — the bars come from the whole file.

The number you would have reported is 95%. The number that matters is 55%.

AND WHERE TO START

Three of the four path failures sit on top of a search failure. Fix the earliest failing step — every step after it is measured on its output.

Rates computed from kestrel-traces.jsonl, not asserted. Change the file and they change — recompute before you quote them.

03.
Benchmarks vs Evals

Evals on your task bring the final choice

Two different questions

A benchmark ranks models. Your eval ranks your options.

A BENCHMARK

“Which model is generally better at *this*?”

IT HOLDS THE TASK FIXED AND VARIES THE MODEL
One task, written by them. Many models. One ranking.

task	theirs
data	theirs — public, fixed, shared by everyone
grader	theirs
the prompt	**fixed, and not yours**

YOUR EVAL

“Does *this build* do *my* job?”

IT HOLDS YOUR TASK FIXED AND VARIES WHAT YOU CONTROL
Your prompt, your context, your retrieval, your tools — and only then the model.

task	yours
data	your own traffic, frozen as a golden set
grader	the rubric you wrote
the prompt	**the largest lever you own**

Same three parts — a task, a transcript, a grader. Different task, different data, different *good*.

A benchmark has never seen your prompt.

Its score is a fact about a model on somebody else's data, with somebody else's prompt. It is not a prediction about your application.

A benchmark is an eval — somebody else's. Same three parts as yours: a task, a transcript, a grader.

Treat the number as a ceiling

Three reasons the gap is wider than it looks.

WHY THE GAP IS WIDER THAN IT LOOKS	THE EVIDENCE	SO WHAT
it may have seen the answers	SWE-Bench: up to **76%** of buggy files identified from the issue text alone, *with no repository at all* — against **53%** on repositories it had not seen.	Read a published score as an **upper bound**, not an estimate.
it measures its data, not yours	General in the marketing, never in the sample. The tasks were chosen by somebody with a different problem.	**Read its task definition** before you believe its ranking.
the board itself is played	One provider tested **27 private variants** before a single public launch. Rational, and it changes what the number means.	You are seeing the **best of twenty-seven**, not the average.

YOU HAVE DONE THIS BEFORE

Session 2's demo 3 was reading these boards critically. **The missing piece then was a golden set of your own.** You have one now, so the reading can finish somewhere.

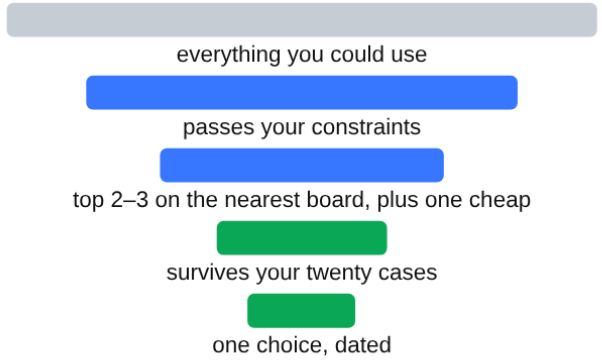
None of this makes a benchmark useless.

It makes it a **filter**, not a verdict.

SWE-Bench Illusion, ICSE-SEIP 2026 (arXiv:2506.12286) · Raji et al., NeurIPS 2021 D&B · Singh et al., The Leaderboard Illusion, NeurIPS 2025 D&B.

How to shortlist two or three

Constraints first. Your twenty cases last.



1. **Constraints first.** Modality · the context length you actually need · latency budget · cost at your volume · where the data is allowed to go · structured output and tool calls. **Most of the list dies here, before any quality question is asked.**
2. **Rank what survives on the board closest to your task.** Not the general one. Open-ended chat preference (**LMArena**), cost and speed (**Artificial Analysis**), coding, agents and tool use are different boards and they do not agree.
3. **Read that board's task definition.** If its tasks do not look like yours, its ranking is not about you.
4. **Take the top two or three — plus one cheap model as a baseline.** The cheap one wins more often than people expect, and the only way to find out is to run it.
5. **Run your twenty cases against all of them.** Same prompt, same golden set. Record cost and latency while you are there.
6. **Decide, and write the date on the decision.** It has a shelf life. Models change under you and so does your traffic.

AND THE DECISION LOOKS LIKE THIS

	YOUR 20 CASES	COST / 1 000 RUNS	P95 LATENCY
A — top of the board	**18 / 20**	€4.80	6.1 s
B	17 / 20	€1.90	3.4 s
C — the cheap baseline	16 / 20	**€0.35**	**1.2 s**

A is two cases better and fourteen times the price.

Which two cases? Go and read them. If they are the two that matter, pay. If they are not, ship C. **That is the decision, and no leaderboard could have made it for you.**

Record the date beside it. In six months the boards will have moved twice, and this table is how you redo the whole exercise in an hour.

Boards named as of October 2026 and they move — re-check before reuse. Session 2's model table is the starting list, prices included.

03.
So what?

Takeaways, then an hour on your build

What to take away

Five sentences, and one table you should have filled in

- With no answer key you write a rubric — and a rubric is the product specification nobody wrote down.
- One dimension per judge call, an explicit way out, and the evidence quoted before the verdict.
- A judge is evidence only after you measure its agreement with a human on held-out labels.
- Grade every step of the trace. On tonight's twenty, the answer passed 95% of the time and only 55% of the runs were right.
- A benchmark is somebody else's golden set. It shortlists two or three models; your twenty cases decide.

For the project: one golden set, one number, and the failure taxonomy it came from.

References

Sources used in this session

- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS D&B.
- Husain, H. Creating a LLM-as-a-Judge That Drives Business Results — hamel.dev/blog/posts/llm-judge/
- Anthropic Engineering (2026). Demystifying Evals for AI Agents (9 Jan 2026).
- Es et al. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation — docs.ragas.io
- The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. ICSE-SEIP 2026.
- Raji et al. (2021). AI and the Everything in the Whole Wide World Benchmark. NeurIPS D&B.
- Singh et al. (2025). The Leaderboard Illusion. NeurIPS D&B — arxiv.org/abs/2504.20879
- Huang et al. (2024). LLMs Cannot Self-Correct Reasoning Yet. ICLR.
- Yao et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction. Sierra.

Where you should be tonight

One week in, one week left. Four things should exist.

☐	**a one-sentence problem statement**	a stranger understands it	**if not:** write it before you touch a tool
☐	**a running skeleton**	ugly, incomplete, but it loads and does one thing end to end	**if not:** this is your whole hour
☐	**twenty real inputs**	collected or written, in a sheet	**if not:** this is your second hour, this week
☐	**one failure you have actually seen**	and can describe in a sentence	**if not:** you have not run it enough

THE RUBRIC HAS NOT CHANGED

problem framing **10** · prompt & context quality **30** · iteration evidence **10** · evaluation **20** · working demo **20** · deliverables quality **10**

Full brief on Moodle. It has not moved and it will not.

Nobody is behind for having an ugly skeleton.

People are behind for having a beautiful plan and no skeleton.

Nobody is behind for having an ugly skeleton. People are behind for having a beautiful plan and no skeleton.

Three things that sink a project

All three are recoverable tonight, and not next Thursday

SCOPE

SYMPTOM

the demo needs three screens and a login

CAUSE

you designed a product

FIX

delete features until one screen does one job, then stop deleting

NO EVALUATION

SYMPTOM

“it works well”, with no number behind it

CAUSE

you left evals to the end, where they are worth nothing

FIX

twenty inputs in a sheet tonight — you have the method

THE LAST WEEKEND

SYMPTOM

nothing committed, everyone “thinking about it”

CAUSE

three people waiting on each other

FIX

split it now: one builds, one collects inputs, one writes

All three are recoverable in the next hour. None of them is recoverable on the nineteenth.

Karpathy coined “vibe coding” in Feb 2025 and framed it as suited to “throwaway weekend projects”. Take the scope advice; you still owe the report.

Stuck? Try in this order

Leave this up. Work through it before you put a hand up.

1. **Read the actual error out loud** — to the person next to you
2. **Start a new chat** — half of all stuck sessions are a polluted context window — Session 3
3. **Ask the tool to explain what it just did** — before you ask it to fix it
4. **Cut the feature** — one screen, eight days
5. **Then put a hand up**

Steps 1–4 solve most of them. And all four are things you will do alone next week, when nobody is in the room.

Steps 1–4 solve most of them, and all four are things you will do alone next week when nobody is in the room.

NOVA IMS INFORMATION MANAGEMENT SCHOOL

See you next week.

NOVA IMS Accreditations and Certifications

eduniversal · QS STARS · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · apcer · IQNet

Co-financed by

PORTUGAL 2030 · Lisboa 2030 Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU